Lab 05 — SVM and Neural Networks, Built in JAX and Keras

Fundamentos de Aprendizagem Automática

Mário Antunes

2026

Overview

Class 05 · Topic 4. Lecture: `slides/05_svm_neural_networks.pdf`. Demo notebooks in `notebooks/05-svm-neural-networks/`: `01_svm.ipynb` and `02_neural_networks.ipynb`.

Notebook for this lab: `notebooks/05-svm-neural-networks/lab05_svm_neural_netw` with helpers in `lab05_utils.py` (same folder, do not edit). Cells marked **TODO** contain `raise NotImplementedError`: replace it with your code. Every other cell is ready, and it draws the plots and prints the comparisons once your functions work.

Solution: `solutions/lab05_svm_neural_networks_solution.ipynb` is the fully solved notebook. Before each function it explains how the solution was reached, and after each experiment it answers this guide's questions with the numbers the run produces. Try every task yourself first.

The rule of this lab. The SVM, the kernel SVM, the SVR and the small network of Part E are *losses written with* `jax.numpy`: the gradient comes from `jax.grad` (through `train`, the Adam optimizer of Class 02). Part F uses **Keras** (the JAX backend), the tool for real networks. The small network of Part E is compared with scikit-learn's `MLPClassifier` and with a Keras network. Every model is compared with its scikit-learn or Keras twin on the **same split and the same scaling**, and the notebook prints the largest difference. A difference is not an error to hide: explain it.

By the end of the lab you should be able to:

1. Write the SVM as a hinge loss plus a penalty, train it with `jax.grad`, and

read the support vectors and the effect of C.

2. Build the RBF and polynomial kernels with broadcasting, train a kernel SVM in its representer form, and choose C and γ on a log grid, after scaling.
3. Write the ε-insensitive loss of the SVR and see why it resists outliers.
4. Write a multilayer perceptron in JAX (initialization, forward pass, cross-entropy), derive its backward pass by hand, check it against `jax.grad` and compare it with the same network in scikit-learn and Keras.
5. Build and regularize a network with Keras, and compare learners with a paired test that accounts for repeated splits.

Part	Task	twin	Time
A	Data visualization and preparation: breast cancer, digits, moons; scaling on the training part	—	10 min
B	The linear SVM: hinge loss with `jax.grad`, support vectors, C	`SVC(kernel="linear")`	20 min
C	Kernels: RBF and polynomial by broadcasting, the kernel SVM in the representer form, γ, the grid	`SVC`, `rbf_kernel`, `GridSearchCV`	30 min
D	Support vector regression: the ε-insensitive loss	`SVR`, `Linear-Regression`	10 min
E	A multilayer perceptron in JAX: forward pass, loss, backpropagation by hand; the same network in three tools	`jax.grad`, `MLPClassifier`, Keras	30 min

Part	Task	twin	Time
F	Networks with Keras: building, regularizing, early stopping, seeds	`MLPClassifier`	15 min
G	Challenge: which learner? a corrected paired test	`LogisticRegression`, SVC, Keras	home

Setup

```
cd faa
source venv/bin/activate        # see README: make venv, or pip install .
export KERAS_BACKEND=jax        # once, in ~/.bashrc
jupyter lab notebooks/05-svm-neural-networks/lab05_svm_neural_networks.ip
```

The data are in `datasets/` (read with polars, see `datasets/README.md`): **breast cancer** (569 tumours, 30 measurements of the cell nuclei, malignant or benign), **digits** (1797 images of 8x8 pixels, 10 classes), **concrete** (1030 mixtures) and two synthetic **moons**. `stratified_split`, `fit_standardizer` and `standardize` are your Class 03 functions, given ready-made.

Part A — Data Visualization and Preparation

The notebook loads the data, splits them (stratified) and standardizes with the **training** statistics. It shows two of the 30 features, the correlation matrix, a sample of digits and the moons.

1. What is the accuracy of a classifier that always answers “benign”? What does it say about the accuracy you will see in the next parts?
2. The table gives the minimum, maximum and standard deviation of five features. Which feature would decide a Euclidean distance, or a penalty $\|w\|^2$, without scaling? By what factor?
3. Look at the correlation matrix. Which features carry nearly the same information, and what does that do to the individual weights of a linear model?

Part B — The Linear SVM as a Loss

With $y_i \in \{-1, +1\}$ and $f(x) = w^\top x + b$:

$$\mathcal{L}(w, b) = \frac{1}{n} \sum_i \max\left(0,\ 1 - y_i f(x_i)\right) + \frac{\lambda}{2}\|w\|^2, \qquad \lambda = \frac{1}{Cn}.$$

B1. The loss and its support vectors

Implement `hinge_loss(params, X, y, lam)` with `params = {"w", "b"}` (**no gradient formula**), `margins(params, X, y)` $= y_i f(x_i)$ and `support_mask(params, X, y, tol)`: the points with margin $\leq 1 +$ tol. The driver trains for $C \in \{0.01, 0.1, 1, 10\}$ on the breast-cancer data and compares weights, predictions and the number of support vectors with `SVC(kernel="linear")`. Then it draws three values of C on two blobs: the line, the margin lines $f = \pm 1$ and the support vectors.

1. For margins $2, 0.5$ and -1: what is the hinge loss, and what is its subgradient with respect to w? Compare with the squared error $(1 - m)^2$.
2. How does the number of support vectors change with C, and why?
3. At $C = 10$ your weights differ from SVC's by far more than at $C \leq 1$, yet the predictions agree on almost every test point. Why?
4. In the picture, what changes with C: the line, the width of the margin, or the points that matter?

Part C — Kernels

With a kernel k and the Gram matrix $K_{ij} = k(x_i, x_j)$ the SVM in its **representer form** is $f(x) = \sum_j \beta_j\, k(x_j, x) + b$, trained on

$$\mathcal{L}(\beta, b) = \frac{1}{n} \sum_i \max\left(0,\ 1 - y_i f(x_i)\right) + \frac{\lambda}{2}\, \beta^\top K \beta.$$

C1. The kernels and the loss

Implement, **without a loop over the points**, `rbf_kernel(A, B, gamma)` (all pairs, shape `(len(A), len(B))`; hint: $\|a - b\|^2 = \|a\|^2 + \|b\|^2 - 2a^\top b$), `poly_kernel(A, B, degree, coef0)` and `kernel_svm_loss(params, K,`

`y, lam)` with `params = {"beta", "b"}`. The driver compares your Gram matrices with scikit-learn's, trains on the moons, compares with `SVC(kernel="rbf")`, and sweeps γ.

1. What are the diagonal and the range of an RBF Gram matrix? What does a value near 0 mean?
2. Which γ underfits and which overfits? What in the table and in the picture tells you?
3. The grid search on raw and on standardized features gives very different best cells and times. Why? (Use the feature ranges of Part A.)
4. The Gram matrix of n points has n^2 entries. How much memory does it need for $n = 100\,000$ in float64? What does it say about the use of kernel SVMs?

Part D — Support Vector Regression

D1. The loss

Implement `eps_insensitive(r, eps)` $= \max(0, |r| - \varepsilon)$ and `svr_loss(params, X, y, eps, lam)` (the mean of the loss of the residuals $y - (Xw + b)$ plus $\frac{\lambda}{2}\|w\|^2$). The driver fits a noisy line with three gross outliers on the same side by least squares, by your loss and by `SVR(kernel="linear")`.

1. The true line is $y = 2x + 1$. Which fit is closest, and which parameter do the outliers disturb?
2. What does $\varepsilon = 0$ give? Which points are the support vectors of an SVR?

Part E — A Multilayer Perceptron in JAX

A practice exercise: the smallest network, written from the definitions. The parameters are a list `[{"W", "b"}, ...]` (a pytree).

E1. Initialization, forward pass, loss

Implement `init_params(key, sizes)` (He: $W \sim \mathcal{N}(0, 2/n_{\text{in}})$, $b = 0$), `forward(params, X)` (ReLU after every layer but the last; returns the logits) and `cross_entropy(params, X, y)` ($\log \sum_c e^{z_c} - z_y$ with `jax.nn.logsumexp`, the mean over the examples). The driver solves XOR with `[2, 2]` and `[2, 8, 2]`, then trains `[64, 32, 10]` on the digits.

1. Why does the network without a hidden layer fail on XOR, and by how much? What is the best accuracy a line can reach?
2. The loss before training is close to $\log 10$. Why? What would a much larger value tell you?
3. How many parameters has `[64, 32, 10]`? How do the training loss and the test accuracy compare?

E2. Backpropagation by hand

Implement `backward_by_hand(params, X, y)` for the network with **one** ReLU hidden layer, returning a list with the structure of `params`: the gradient of `cross_entropy`. Chain rule: $\partial \mathcal{L}/\partial z_2 = (p - \text{onehot}(y))/n$; a layer $z = hW + b$ sends δ back with $W^\top$ and gives $h^\top \delta$ for its weights; the ReLU passes the gradient where $z_1 > 0$. The driver compares with `jax.grad`.

1. How close is your backward pass? How many forward passes would finite differences need for the same gradient, and why is backpropagation cheaper?

E3. Three implementations of the same network

No code to write. The driver trains the network of E1 (`[64, 32, 10]`) in scikit-learn (`MLPClassifier`) and in Keras (`Sequential`, JAX backend) with the same split, the same scaling, Adam with learning rate 0.01, full batch and 1000 steps. It prints the parameters, the training loss, the test accuracy and the share of test images on which two models agree.

1. Do the three implementations have the same number of parameters? How far apart are their test accuracies, and is that more than the seed-to-seed spread of Part F?
2. Name two things that differ between the three (initialization, regularization, stopping rule, …) and say which one you expect to matter most.

Part F — Networks with Keras

F1. Build the network

Implement `build_keras_mlp(sizes, n_in, n_out, l2, dropout, seed)`: `keras.utils.set_random_seed(seed)`, a `Sequential` with `Input`, one `Dense(size, activation="relu", kernel_regularizer=L2(l2))`

per hidden size (and `Dropout` if requested), and a final `Dense(n_out, activation="softmax")`. The driver trains five configurations on the digits (no regularization, weight decay, dropout, early stopping, dropout and early stopping), compares them with your Part E network and with `MLPClassifier`, plots the validation loss, and trains ten seeds.

1. Without regularization the training accuracy is 1.000. What do the validation loss and early stopping tell you?
2. Which configuration is best? Is the difference larger than the spread over seeds?
3. Keras, plain JAX and scikit-learn implement the same model. What explains the small differences between them?

Part G — Challenge: Which Learner?

Choose a data set of `datasets/` (`german_credit`, `wine_quality`, `heart_disease`, `spambase`), prepare it with polars (missing values, one-hot encoding, a binary target) and compare **logistic regression**, the **RBF SVM** and a **Keras MLP** on 10 stratified 75/25 splits, with the standardization fitted on the training part.

G1. The corrected test

Implement `corrected_ttest(a, b, ratio)`: the paired t statistic with the variance $\left(\frac{1}{J} + \frac{n_{\text{test}}}{n_{\text{train}}}\right) s_d^2$ and the two-sided p-value (`scipy.stats.t`). The notebook runs the comparison on the wine data.

1. Why is the ordinary paired t-test too optimistic for repeated random splits?
2. Which differences are significant, and what do you conclude between the SVM and the MLP? What would decide between them in practice?
3. Repeat on another data set. Does the ranking change? Does the cost (training time) change the choice?

Going further. Tune C and γ (or the width and the learning rate) *inside* each split with a nested cross-validation (Class 01) and see whether the differences survive. Or train the Part E network with minibatches and compare its learning curve with Keras's.